

Curso Profesional de JasperReports 6.20.0 Community

Módulo 3 — Conexión a datos

TEORÍA

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Maven · SQLite · CSV · XML · JSON · Proyecto EditorialReports

Curso Profesional de JasperReports 6.20.0 Community

Módulo 3 - Conexión a datos - TEORÍA

Proyecto: EditorialReports

Autor: JAIME GALLO

Baseline: Jaspersoft Studio 6.20.0 Community + JasperReports Library 6.20.0 + Java 8 + Maven

Puntos incluidos

- 3.1 Bases de datos y JDBC
- 3.2 Ficheros CSV
- 3.3 Ficheros XML
- 3.4 Ficheros JSON
- 3.5 Consultas SQL
- 3.6 Fields

Estado del proyecto al inicio del módulo

M3 comienza desde el checkpoint validado M2/2.6. Se conservan EditorialReports, EditorialReportsJava, el informe conceptual, las clases Libro, CatalogoDataSource y GeneradorInformeConcepto, los recursos gráficos y la baseline Java 8/JasperReports 6.20.0. El módulo añade fuentes JDBC, CSV, XML y JSON y termina con consultas SQL agregadas y una revisión profesional del contrato de campos.

Correcciones técnicas consolidadas en esta edición

Esta edición separa el origen docente recibido de la solución ejecutable final. Se han corregido: la versión Maven real del driver Xerial (3.44.0.0), el mapeo XPath mediante `net.sf.jasperreports.xpath.field.expression`, la selección explícita `JsonDataSource(..., "autores")`, la sintaxis del lenguaje JSON de JasperReports frente a JSONPath externo, la parametrización SQL con `$P{}` mediante `PreparedStatement`, el soporte de `RIGHT/FULL OUTER JOIN` en SQLite 3.44, el tratamiento de fechas almacenadas como TEXT, el uso de `textAdjust="StretchHeight"` y los nulos del informe final mediante `LEFT JOIN`.

Punto 3.1 — Bases de datos y JDBC

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.1 — Bases de datos y JDBC

Objetivos de aprendizaje

- Comprender el papel de JDBC como interfaz de conexión entre JasperReports y las bases de datos.
- Instalar y configurar el driver JDBC de SQLite en el proyecto Java y en Jaspersoft Studio.
- Crear una base de datos SQLite con la tabla de libros del proyecto EditorialReports.
- Configurar un adaptador JDBC en Jaspersoft Studio y asociarlo al informe.
- Declarar una consulta SQL en el JRXML y verificar su ejecución desde la previsualización.
- Ejecutar el informe desde código Java con una conexión JDBC.
- Documentar la conexión a la base de datos en el proyecto.

Parte teórica

Bloque 1 — JDBC como interfaz de conexión

JDBC (Java Database Connectivity) es la interfaz estándar de Java para conectarse a bases de datos relacionales. Define un conjunto de clases e interfaces que permiten establecer una conexión, ejecutar consultas y procesar resultados con independencia del motor de base de datos subyacente. JasperReports utiliza JDBC como uno de sus mecanismos principales para obtener datos: el motor recibe una conexión JDBC activa, ejecuta la consulta declarada en el JRXML y recorre el `ResultSet` resultante registro a registro. Esta integración permite que un informe se alimente directamente de una base de datos sin necesidad de escribir una clase `JRDataSource` personalizada. La conexión se pasa al motor mediante el método `fillReport` que acepta un objeto `java.sql.Connection`.

JAVA

```
Connection conexion = DriverManager.getConnection(
    "jdbc:sqlite:../EditorialReportsJava/data/editorial.db");
```

Línea 1	<code>Connection conexion =</code> → declara una variable de tipo <code>java.sql.Connection</code> , que representa la conexión activa con la base de datos.
Línea 1 (continuación)	<code>DriverManager.getConnection(</code> → invoca al gestor de drivers de JDBC para establecer la conexión. El gestor localiza el driver adecuado según la URL.
Línea 2	<code>"jdbc:sqlite:../EditorialReportsJava/data/editorial.db"</code> → URL de conexión. El prefijo <code>jdbc:sqlite:</code> indica el motor SQLite y la parte final indica la ruta del archivo de base de datos.
Línea 2 (continuación)	<code>);</code> → cierra la llamada y asigna el resultado a la variable.

La URL de conexión JDBC tiene una estructura específica para cada motor. En SQLite, el formato es `jdbc:sqlite:ruta/al/archivo.db`. En MySQL, el formato es `jdbc:mysql://host:puerto/base`. En PostgreSQL, el formato es `jdbc:postgresql://host:puerto/base`. La elección del motor determina el driver que debe estar en el classpath y la URL que debe utilizarse. En el proyecto EditorialReports se utiliza SQLite porque no requiere un servidor de base de datos independiente: la base de datos es un único archivo que se distribuye con el proyecto. Esta característica facilita la portabilidad y la reproducibilidad del entorno de desarrollo.

TEXT

COMPONENTES DE UNA CONEXIÓN JDBC

Driver JDBC	→ Implementación específica del motor (sqlite-jdbc-3.44.0.0.jar para SQLite)
URL de conexión	→ Identifica el motor y la base de datos (jdbc:sqlite:../EditorialReportsJava/data/editorial.db)
Usuario y contraseña	→ Credenciales de acceso (SQLite no las requiere)
Objeto Connection	→ Conexión activa con la base de datos

	(<code>java.sql.Connection</code>)
Objeto <code>Statement</code>	→ Sentencia SQL que se va a ejecutar (<code>java.sql.Statement</code>)
Objeto <code>ResultSet</code>	→ Resultado de la consulta (<code>java.sql.ResultSet</code>)

Qué representa el diagrama: los componentes que intervienen en una conexión JDBC. El driver, la URL y las credenciales son la configuración. El `Connection`, el `Statement` y el `ResultSet` son los objetos que se crean durante la ejecución.

Por qué es relevante: permite comprender qué componente falla cuando aparece un error de conexión. Un error de driver indica que falta el JAR. Un error de URL indica que la ruta es incorrecta. Un error de credenciales indica que el usuario o la contraseña no son válidos.

Bloque 2 — El driver JDBC de SQLite

El driver JDBC de SQLite se distribuye como un único archivo JAR denominado `sqlite-jdbc-3.44.0.0.jar`. Este archivo contiene la implementación del driver y las bibliotecas nativas de SQLite para los sistemas operativos soportados. El driver se registra automáticamente en el `DriverManager` cuando el JAR está en el classpath, por lo que no es necesario invocar `Class.forName` en el código. La versión 3.44.0 es la utilizada en este curso y es compatible con Java 8 y versiones superiores. El JAR debe añadirse al classpath del proyecto Java y a la configuración del adaptador JDBC en Jaspersoft Studio.

`sqlite-jdbc-3.44.0.0.jar`

TEXT

Línea 1 `sqlite-jdbc-3.44.0.0.jar` → archivo JAR que contiene el driver JDBC de SQLite. Debe copiarse a la carpeta `lib` del proyecto Java y referenciarse en el Build Path.

La base de datos SQLite es un único archivo con extensión `.db` que contiene todas las tablas, índices y datos. El archivo se crea automáticamente al establecer la primera conexión si no existe. Esta característica simplifica la creación de la base de datos: basta con conectarse a una ruta inexistente y ejecutar las sentencias `CREATE TABLE` necesarias. La base de datos del proyecto `EditorialReports` se denomina `editorial.db` y reside en la carpeta `data` del proyecto Java. La carpeta `data` es hermana de `lib` y `src` y forma parte del proyecto versionable.

```

EditorialReportsJava/
├── lib/
│   ├── jasperreports-6.20.0.jar
│   ├── commons-digester-2.1.jar
│   ├── commons-collections-3.2.2.jar
│   ├── commons-logging-1.2.jar
│   ├── ecj-3.24.0.jar
│   └── sqlite-jdbc-3.44.0.0.jar           (nuevo)
├── data/
│   └── editorial.db                     (base de datos)
└── src/
    ├── GeneradorInformeConcepto.java
    ├── Libro.java
    └── CatalogoDataSource.java

```

TEXT

Qué representa el diagrama: la estructura del proyecto Java tras añadir la carpeta `data` con la base de datos SQLite y el JAR del driver en la carpeta `lib`.

Por qué es relevante: permite localizar los archivos necesarios para la conexión JDBC y mantener la coherencia con la estructura del proyecto.

Bloque 3 — Configuración del adaptador JDBC en Jaspersoft Studio

Jaspersoft Studio permite configurar adaptadores de datos JDBC desde el panel `Repository Explorer`. Un adaptador JDBC almacena la URL de conexión, el driver, el usuario y la contraseña para que el entorno pueda conectarse a la base de datos durante la previsualización. El adaptador se asocia al informe mediante la propiedad `com.jaspersoft.studio.data.defaultdataadapter` del `JRXML`. Esta asociación permite que el botón `Preview` ejecute la consulta SQL declarada en el informe y muestre el resultado con datos reales. El adaptador se configura una sola vez y se reutiliza en todos los informes del proyecto.

```

Repository Explorer > Data Adapters > Create Data Adapter
├── Database JDBC Connection
├── Name: SQLiteEditorial
├── JDBC Driver: org.sqlite.JDBC
├── JDBC URL: jdbc:sqlite:../EditorialReportsJava/data/editorial.db
├── Username: (vacío)
├── Password: (vacío)
└── Test Connection → OK

```

TEXT

Línea 1	Repository Explorer > Data Adapters > Create Data Adapter → ruta de menú para crear un nuevo adaptador de datos.
Línea 2	Database JDBC Connection → tipo de adaptador que se va a crear.
Línea 3	Name: SQLiteEditorial → nombre del adaptador en el panel Repository Explorer.
Línea 4	JDBC Driver: org.sqlite.JDBC → nombre completo de la clase del driver.
Línea 5	JDBC URL: jdbc:sqlite:../EditorialReportsJava/data/editorial.db → URL de conexión a la base de datos.
Línea 6-7	Username: (vacío) y Password: (vacío) → SQLite no requiere credenciales.
Línea 8	Test Connection → OK → botón que verifica que la conexión se establece correctamente.

El adaptador JDBC se asocia al informe mediante la propiedad correspondiente en el JRXML. La propiedad se añade automáticamente cuando se crea el informe desde el asistente o se selecciona el adaptador desde el panel Properties. Si la propiedad no está presente, el botón Preview muestra un diálogo para seleccionar el adaptador en cada ejecución. La asociación persistente simplifica el trabajo y evita tener que seleccionar el adaptador en cada previsualización. La propiedad se declara con el nombre del adaptador como valor.

XML

```
<property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/>
```

Línea 1 <property name="com.jaspersoft.studio.data.defaultdataadapter" value="SQLiteEditorial"/> → asocia el adaptador SQLiteEditorial al informe. El botón Preview ejecuta la consulta SQL contra la base de datos especificada en el adaptador.

Bloque 4 — La consulta SQL en el JRXML

La consulta SQL que alimenta el informe se declara en el JRXML mediante el elemento queryString. Este elemento contiene la sentencia SQL que el motor ejecuta contra la conexión JDBC para obtener los registros. El atributo language del elemento indica el lenguaje de la consulta, que en este caso es sql. El contenido se encierra en un bloque CDATA para evitar conflictos con los caracteres especiales del XML. La consulta puede incluir parámetros mediante la sintaxis \$P{ }, aunque esta característica se estudia en el Módulo 4. En este punto la consulta es estática.

XML

```
<queryString language="sql">
  <![CDATA[SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, CASE WHEN disponible=1 THEN 1 ELSE 0 END AS
disponible FROM libros ORDER BY titulo]]>
</queryString>
```

Línea 1 <queryString language="sql"> → declara la consulta SQL del informe. El atributo language="sql" indica al motor que debe interpretar el contenido como una sentencia SQL.

Línea 2 <![CDATA[SELECT titulo, precio, paginas, fecha_publicacion AS fechaPublicacion, CASE WHEN disponible=1 THEN 1 ELSE 0 END AS disponible FROM libros ORDER BY titulo]]> → sentencia SQL que selecciona las columnas necesarias de la tabla libros y las ordena por título.

Línea 3 </queryString> → cierra la declaración de la consulta.

La correspondencia entre las columnas del ResultSet y los campos del JRXML se establece por nombre. El motor toma cada columna del resultado y la mapea al campo cuyo nombre coincide. Si una columna del ResultSet no tiene un campo correspondiente en el JRXML, se ignora. Si un campo del JRXML no tiene una columna correspondiente en el ResultSet, el motor lanza Field not found al resolver la expresión que lo referencia. La coherencia entre los nombres de las columnas y los nombres de los campos es condición necesaria para que la resolución funcione.

TEXT

CORRESPONDENCIA ENTRE COLUMNAS Y CAMPOS

ResultSet de la consulta SQL

titulo
precio
paginas
fechaPublicacion
disponible

Campos del JRXML

<field name="titulo" class="java.lang.String"/>
<field name="precio" class="java.lang.Double"/>
<field name="paginas" class="java.lang.Integer"/>
<field name="fechaPublicacion" class="java.lang.String"/>
<field name="disponible" class="java.lang.Boolean"/>

Qué representa el diagrama: la correspondencia por nombre entre las columnas del `ResultSet` y los campos declarados en el JRXML. Los nombres deben coincidir exactamente.

Por qué es relevante: permite diagnosticar errores de resolución de campos cuando se trabaja con una base de datos. Si un campo no se resuelve, la causa suele ser una discrepancia entre el nombre de la columna y el nombre del campo.

Bloque 5 — Ejecución del informe con una conexión JDBC

La ejecución del informe con una conexión JDBC sustituye la fuente de datos personalizada por una conexión activa. El método `JasperFillManager.fillReport` acepta un objeto `java.sql.Connection` como tercer argumento en lugar de un `JRDataSource`. El motor ejecuta la consulta declarada en el JRXML contra la conexión y recorre el `ResultSet` resultante. Esta forma de ejecución simplifica el código porque no es necesario escribir una clase que implemente `JRDataSource`. La contrapartida es que el informe queda acoplado al esquema de la base de datos y la consulta SQL forma parte del JRXML.

JAVA

```
Connection conexion = DriverManager.getConnection(
    "jdbc:sqlite:../EditorialReportsJava/data/editorial.db");

JasperPrint documento = JasperFillManager.fillReport(
    "reports/informe_concepto.jasper",
    new HashMap<String, Object>(),
    conexion);

conexion.close();
```

Línea 1-2	<code>Connection conexion = DriverManager.getConnection("jdbc:sqlite:../EditorialReportsJava/data/editorial.db");</code> → establece la conexión con la base de datos SQLite.
Línea 4	<code>JasperPrint documento =</code> → declara la variable que recibirá el documento en memoria.
Línea 4 (continuación)	<code>JasperFillManager.fillReport(</code> → invoca al motor de llenado.
Línea 5	<code>"reports/informe_concepto.jasper",</code> → ruta del artefacto compilado.
Línea 6	<code>new HashMap<String, Object>(),</code> → mapa de parámetros vacío.
Línea 7	<code>conexion);</code> → conexión JDBC que el motor utiliza para ejecutar la consulta. El método cierra el <code>ResultSet</code> internamente pero no cierra la conexión.
Línea 9	<code>conexion.close();</code> → cierra la conexión una vez finalizado el llenado. La conexión debe cerrarse siempre para liberar los recursos del sistema.

La conexión JDBC debe cerrarse siempre después de su uso. Si se olvida, la conexión permanece abierta hasta que el programa termina y consume recursos del sistema. En SQLite, la conexión abierta bloquea el archivo de base de datos e impide que otras aplicaciones lo modifiquen. La buena práctica consiste en cerrar la conexión en un bloque `finally` o en un bloque `try-with-resources` que garantice el cierre incluso si se produce una excepción. El bloque `try-with-resources` es la forma más limpia y la que se utiliza en el proyecto `EditorialReports`.

JAVA

```
try (Connection conexion = DriverManager.getConnection("jdbc:sqlite:../EditorialReportsJava/data/editorial.db")) {
    JasperPrint documento = JasperFillManager.fillReport(
        "reports/informe_concepto.jasper",
        new HashMap<String, Object>(),
        conexion);
    JasperExportManager.exportReportToPdfFile(documento, "output/informe_concepto.pdf");
}
```

Línea 1	<code>try (Connection conexion = DriverManager.getConnection("jdbc:sqlite:../EditorialReportsJava/data/editorial.db")) {</code> → abre un bloque <code>try-with-resources</code> que garantiza el cierre automático de la conexión al finalizar el bloque, incluso si se produce una excepción.
Línea 2-5	<code>JasperFillManager.fillReport(...)</code> llena el informe con la conexión.
Línea 6	<code>JasperExportManager.exportReportToPdfFile(...)</code> exporta el informe a PDF.
Línea 7	<code>}</code> → cierra el bloque <code>try-with-resources</code> y la conexión se cierra automáticamente.

Resumen rápido de la teoría

- JDBC es la interfaz estándar de Java para conectarse a bases de datos relacionales.
 - JasperReports utiliza JDBC para ejecutar la consulta declarada en el JRXML.
 - El driver JDBC de SQLite se distribuye como `sqlite-jdbc-3.44.0.0.jar`.
 - Jaspersoft Studio permite configurar adaptadores JDBC para la previsualización.
 - La consulta SQL se declara en el elemento `queryString` del JRXML.
 - La correspondencia entre columnas y campos se establece por nombre.
 - La ejecución desde Java utiliza `JasperFillManager.fillReport` con una conexión JDBC.
 - La conexión debe cerrarse siempre con un bloque `try-with-resources` o `finally`.
-
-

Punto 3.2 — Ficheros CSV

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.2 — Ficheros CSV

Objetivos de aprendizaje

- Comprender el formato CSV y su papel como fuente de datos ligera.
- Configurar un adaptador CSV en Jaspersoft Studio y asociarlo a un informe.
- Declarar los campos del informe a partir de las columnas del archivo CSV.
- Leer un archivo CSV desde código Java sin depender de Jaspersoft Studio.
- Combinar datos de un archivo CSV con datos de la base de datos en un mismo informe.
- Documentar el uso de archivos CSV en el proyecto EditorialReports.

Parte teórica

Bloque 1 — El formato CSV como fuente de datos

CSV (Comma-Separated Values) es un formato de texto plano que representa una tabla con filas y columnas. Cada línea del archivo corresponde a una fila y las columnas se separan por un delimitador, habitualmente una coma. La primera línea puede contener los nombres de las columnas o puede contener directamente el primer registro. El formato no tiene un estándar formal único, por lo que cada sistema define sus propias convenciones sobre el delimitador, la presencia de cabecera y el carácter de escape. A pesar de esta falta de estandarización, el CSV es el formato de intercambio más universal: cualquier hoja de cálculo, cualquier base de datos y cualquier lenguaje de programación pueden leerlo y escribirlo. En el ámbito editorial, el CSV se utiliza para intercambiar catálogos con distribuidores, para exportar listados desde aplicaciones heredadas y para recibir datos de proveedores externos.

CSV

```

titulo,autor,precio,paginas,fecha_publicacion,disponible
Cien años de soledad,Gabriel García Márquez,19.95,471,1967-06-05,true
Rayuela,Julio Cortázar,22.50,736,1963-06-28,true
La ciudad y los perros,Mario Vargas Llosa,18.75,432,1963-10-15,true
Pedro Páramo,Juan Rulfo,15.90,132,1955-03-01,true

```

Línea 1	titulo,autor,precio,paginas,fecha_publicacion,disponible → línea de cabecera con los nombres de las seis columnas. El delimitador es la coma.
Línea 2	Cien años de soledad,Gabriel García Márquez,19.95,471,1967-06-05,true → primer registro. Cada valor corresponde a una columna de la cabecera.
Línea 3	Rayuela,Julio Cortázar,22.50,736,1963-06-28,true → segundo registro.
Línea 4	La ciudad y los perros,Mario Vargas Llosa,18.75,432,1963-10-15,true → tercer registro.
Línea 5	Pedro Páramo,Juan Rulfo,15.90,132,1955-03-01,true → cuarto registro.

Las ventajas del CSV son su simplicidad, su universalidad y su ligereza. Un archivo CSV puede abrirse con cualquier editor de texto, compararse con cualquier herramienta de diferencias y versionarse sin problemas en un sistema de control de versiones. La contrapartida es que el formato no incluye tipos de datos: todos los valores son cadenas y es el lector quien debe convertirlos al tipo adecuado. Un precio se lee como la cadena "19.95" y debe convertirse a `Double` para poder formatearlo. Una fecha se lee como la cadena "1967-06-05" y debe convertirse a `Date` para poder aplicar un patrón. Esta conversión es responsabilidad de la fuente de datos y no del motor de JasperReports. La simplicidad del formato exige un trabajo adicional en la capa de lectura.

TEXT

VENTAJAS Y DESVENTAJAS DEL CSV

Ventajas:

- Texto plano, legible por humanos
- Compatible con cualquier herramienta
- Ligero y portable
- Fácil de versionar
- No requiere servidor

Desventajas:

- Sin tipos de datos: todo es cadena

- Sin esquema: la estructura debe conocerse
- Sin soporte para relaciones
- Sin soporte para transacciones
- Delimitadores y escapes variables según el emisor

Qué representa el diagrama: las ventajas y desventajas del formato CSV. La ausencia de tipos y de esquema son las limitaciones principales.

Por qué es relevante: permite decidir cuándo conviene utilizar un CSV y cuándo conviene una base de datos. Los CSV son adecuados para datos simples y portables. Las bases de datos son adecuadas para datos complejos con relaciones y transacciones.

Bloque 2 — Lectura de CSV en Jaspersoft Studio

Jaspersoft Studio permite configurar adaptadores de datos que leen archivos CSV y los convierten en fuentes de datos para el motor de JasperReports. El adaptador se configura desde el panel Repository Explorer con la opción CSV File Data Source. En el asistente se especifica la ruta del archivo, el delimitador, la presencia de cabecera y el carácter de escape. El entorno lee el archivo y deduce los campos a partir de la cabecera o de la primera línea. El adaptador se asocia al informe mediante la propiedad `com.jaspersoft.studio.data.defaultdataadapter`, igual que el adaptador JDBC. La asociación permite que el botón Preview ejecute el informe con los datos del CSV.

TEXT

```
Repository Explorer > Data Adapters > Create Data Adapter
→ CSV File Data Source
→ Name: CatalogoCSV
→ File: data/catalogo.csv
→ Charset: UTF-8
→ Delimiter: ,
→ Use First Row as Column Names: marcada
→ Test Connection → OK
```

Línea 1	Repository Explorer > Data Adapters > Create Data Adapter → ruta de menú para crear un nuevo adaptador.
Línea 2	CSV File Data Source → tipo de adaptador CSV.
Línea 3	Name: CatalogoCSV → nombre del adaptador en el panel Repository Explorer.
Línea 4	File: data/catalogo.csv → ruta del archivo CSV. La ruta es relativa al directorio de trabajo del entorno de diseño.
Línea 5	Charset: UTF-8 → codificación de caracteres del archivo. UTF-8 admite todos los caracteres del español.
Línea 6	Delimiter: , → carácter delimitador entre columnas.
Línea 7	Use First Row as Column Names: marcada → indica que la primera línea del archivo contiene los nombres de las columnas.
Línea 8	Test Connection → OK → botón que verifica que el archivo se lee correctamente.

JRCsvDataSource trabaja sobre un flujo/lector CSV y avanza registro a registro cuando el motor llama a `next()`; no necesita materializar previamente una colección de mapas. Con `setUseFirstRowAsHeader(true)`, la primera fila define los nombres de columna y `getFieldValue` resuelve cada field por ese nombre. Los valores parten de texto y `JRAbstractTextDataSource` puede convertirlos al tipo declarado cuando dispone de la configuración de formato necesaria.

TEXT

LECTURA DEL CSV POR EL ADAPTADOR

```
Archivo CSV:
  titulo,autor,precio,paginas
  Cien años de soledad,Gabriel García Márquez,19.95,471

Lectura secuencial:
  next() → fila actual: Cien años de soledad, Gabriel García Márquez, 19.95, 471

Con la cabecera activada:
  ${titulo} → columna "titulo" de la fila actual
  ${precio} → columna "precio" de la fila actual

El origen es textual. La fuente puede convertir al tipo Java declarado
si se configuran los formatos correspondientes.
```

Qué representa el diagrama: el recorrido secuencial del CSV por JRCsvDataSource. La cabecera proporciona los nombres de columna y el motor obtiene los valores de la fila actual a través de los fields.

Por qué es relevante: permite comprender por qué los valores de un CSV se comportan inicialmente como cadenas y por qué la conversión de tipos es necesaria para aplicar patrones numéricos o de fecha.

Bloque 3 — Lectura de CSV desde código Java

La lectura de un archivo CSV desde código Java se realiza con la clase `net.sf.jasperreports.engine.data.JRCsvDataSource`. Esta clase lee el archivo, interpreta la cabecera y construye una fuente de datos que el motor puede recorrer. El constructor puede recibir un archivo, una URL, un `InputStream` o un lector. La codificación puede fijarse con el constructor que recibe `charsetName`; `setFieldDelimiter` configura el delimitador y `setUseFirstRowAsHeader` indica que la primera fila contiene los nombres de las columnas. La fuente de datos se pasa al motor de llenado como tercer argumento de `fillReport`. La clase `JRCsvDataSource` forma parte de la biblioteca `JasperReports` y no requiere dependencias adicionales.

JAVA

```
JRCsvDataSource dataSource =
    new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8");
dataSource.setFieldDelimiter(',');
dataSource.setUseFirstRowAsHeader(true);
```

Líneas 1-2	<code>new JRCsvDataSource(new File("data/catalogo.csv"), "UTF-8")</code> → construye la fuente CSV y fija explícitamente la codificación UTF-8 en el constructor.
Línea 3	<code>dataSource.setFieldDelimiter(',');</code> → configura la coma como delimitador.
Línea 4	<code>dataSource.setUseFirstRowAsHeader(true);</code> → indica que la primera fila contiene los nombres de los campos.

CSV no transporta un esquema de tipos. `JRCsvDataSource` parte de texto y puede convertir valores al tipo declarado si se configuran formatos numéricos/fecha. En este punto se adopta deliberadamente la estrategia más explícita: declarar los campos como `String` y convertir precio y paginas en las expresiones del JRXML. Una alternativa consiste en construir una fuente de datos personalizada que lea el CSV y devuelva los valores ya convertidos. La primera opción es más rápida de configurar. La segunda opción es más flexible y permite reutilizar la lógica de conversión en varios informes. En el proyecto `EditorialReports` se utiliza la primera opción para los informes que se alimentan directamente del CSV.

TEXT

CONVERSIÓN DE TIPOS EN LA EXPRESIÓN

```
Campo declarado:
<field name="precio" class="java.lang.String"/>

Conversión en la expresión:
<textFieldExpression>
  <![CDATA[Double.parseDouble($F{precio})]]>
</textFieldExpression>

O con patrón directo si el campo se declara como Double y la fuente
de datos realiza la conversión:
<field name="precio" class="java.lang.Double"/>
```

Qué representa el diagrama: las dos formas de manejar la conversión de tipos: convertir en la expresión o declarar el campo con el tipo final y confiar en la fuente de datos.

Por qué es relevante: permite elegir la estrategia de conversión según el control que se necesite sobre los datos y la reutilización de la lógica.

Bloque 4 — Mapeo de columnas CSV a campos del JRXML

El mapeo entre las columnas del CSV y los campos del JRXML se realiza por nombre. El nombre del campo debe coincidir exactamente con el nombre de la columna del CSV. La coincidencia es sensible a mayúsculas y minúsculas. Un campo declarado como `titulo` se resuelve con la columna `titulo`. Un campo declarado como `Titulo` no se resuelve con la columna `titulo`. Esta regla es la misma que se aplica a los campos de una base de datos. La coherencia de nombres entre el origen y la plantilla es la primera condición para que el informe funcione.

XML

```
<field name="titulo" class="java.lang.String"/>
<field name="autor" class="java.lang.String"/>
<field name="precio" class="java.lang.String"/>
<field name="paginas" class="java.lang.String"/>
<field name="fecha_publicacion" class="java.lang.String"/>
<field name="disponible" class="java.lang.String"/>
```

Línea 1	<code><field name="titulo" class="java.lang.String"/></code> → declara un campo que corresponde a la columna <code>titulo</code> del CSV.
----------------	---

Línea 2	<code><field name="autor" class="java.lang.String"/></code> → declara un campo que corresponde a la columna autor.
Línea 3	<code><field name="precio" class="java.lang.String"/></code> → declara el precio como cadena porque el CSV no distingue tipos. La conversión a Double se realiza en la expresión o mediante el patrón del campo.
Línea 4	<code><field name="paginas" class="java.lang.String"/></code> → declara el número de páginas como cadena.
Línea 5	<code><field name="fecha_publicacion" class="java.lang.String"/></code> → declara la fecha como cadena.
Línea 6	<code><field name="disponible" class="java.lang.String"/></code> → declara la disponibilidad como cadena. La conversión a booleano se realiza en la expresión.

El mapeo puede automatizarse en Jaspersoft Studio. Cuando se crea un informe a partir de un adaptador CSV, el entorno lee la cabecera del archivo y genera automáticamente las declaraciones de campo con el tipo `java.lang.String`. Esta generación automática evita tener que escribir las declaraciones manualmente y garantiza la coherencia entre los nombres. La operación inversa, es decir, modificar los tipos de los campos generados, se realiza manualmente desde el panel Outline o desde la vista Source. La generación automática es una de las ventajas de Jaspersoft Studio frente a la edición manual del JRXML.

TEXT

GENERACIÓN AUTOMÁTICA DE CAMPOS DESDE UN ADAPTADOR CSV

1. Crear el adaptador CSV en el Repository Explorer.
2. Crear un nuevo informe Jasper Report.
3. Seleccionar el adaptador CSV en el asistente.
4. El asistente lee la cabecera del CSV.
5. El asistente genera las declaraciones `<field>` automáticamente.
6. Los campos se declaran con `class="java.lang.String"`.
7. El usuario puede modificar los tipos manualmente después.

Qué representa el diagrama: el proceso de generación automática de campos a partir de un adaptador CSV. El asistente lee la cabecera y crea las declaraciones correspondientes.

Por qué es relevante: permite aprovechar la generación automática para evitar errores de tipeo y garantizar la coherencia entre los nombres.

Bloque 5 — Combinación de CSV con otras fuentes de datos

Un informe puede combinar datos de varias fuentes mediante subreportes o mediante tablas. La combinación permite enriquecer un informe que se alimenta de un CSV con datos que residen en una base de datos, o viceversa. Un ejemplo típico es un informe de catálogo que se alimenta de un CSV con los títulos y precios, y que incluye un subreporte que consulta la base de datos para obtener los datos del autor. Los subreportes se estudian en el Módulo 5, pero la combinación de fuentes es una técnica habitual que conviene conocer desde el inicio. La clave consiste en pasar los parámetros necesarios al subreporte y en configurar una fuente de datos distinta para cada subreporte.

TEXT

COMBINACIÓN DE CSV Y BASE DE DATOS

```
Informe principal (alimentado por CSV):
├── Título del libro (columna del CSV)
├── Precio (columna del CSV)
└── Subreporte (alimentado por JDBC):
    ├── Datos del autor (consulta SQL)
```

El subreporte recibe el nombre del autor como parámetro.
El motor ejecuta el subreporte para cada registro del CSV.

Qué representa el diagrama: la estructura de un informe que combina un CSV en el nivel principal con una base de datos en un subreporte. El subreporte recibe un parámetro del nivel principal.

Por qué es relevante: permite ampliar el alcance del informe sin renunciar a la simplicidad del CSV. Los datos simples residen en el CSV y los datos complejos en la base de datos.

La combinación de fuentes también puede realizarse en el programa Java que genera el informe. El programa puede leer el CSV, construir una lista de objetos y pasarla como fuente de datos al motor. Esta aproximación permite combinar datos de varias fuentes en memoria antes de pasarlos al motor. La lista de objetos puede construirse a partir de un CSV, de una consulta SQL y de una llamada a un servicio externo. El motor recibe una única fuente de datos con todos los registros combinados. Esta técnica es útil cuando la combinación no puede expresarse como una consulta SQL o cuando los datos proceden de fuentes heterogéneas.

JAVA

```
List<Libro> libros = new ArrayList<Libro>();

try (BufferedReader br = new BufferedReader(new FileReader("data/catalogo.csv"))) {
    String linea;
    boolean primera = true;
    while ((linea = br.readLine()) != null) {
```

```

        if (primera) {
            primera = false;
            continue;
        }
        String[] campos = linea.split(",");
        libros.add(new Libro(
            campos[0],
            Double.parseDouble(campos[2]),
            Integer.parseInt(campos[3]),
            Integer.parseInt(campos[4].substring(0, 4)),
            Boolean.valueOf(campos[5])));
    }
}

JRBeanCollectionDataSource dataSource = new JRBeanCollectionDataSource(libros);

```

Línea 1	<code>List<Libro> libros = new ArrayList<Libro>();</code> → declara la lista con la misma clase Libro utilizada por el proyecto.
Líneas 3-12	abren <code>data/catalogo.csv</code> , omiten la cabecera y recorren las filas. <code>split(",")</code> es suficiente para el CSV didáctico de este curso, cuyos valores no contienen comas escapadas; para CSV generales conviene utilizar <code>JRCSVDataSource</code> o un parser CSV.
Líneas 13-18	construyen Libro con la firma real de la clase heredada de M2: título, precio, páginas, año de publicación y disponibilidad.
Línea 21	<code>JRBeanCollectionDataSource</code> adapta la lista de beans al contrato <code>JRDataSource</code> .

Resumen rápido de la teoría

- CSV es un formato de texto plano que representa tablas con filas y columnas.
- Jaspersoft Studio permite configurar adaptadores CSV para la previsualización.
- La clase `JRCSVDataSource` permite leer CSV desde código Java.
- Los valores del CSV se leen como cadenas y deben convertirse al tipo adecuado.
- El mapeo entre columnas y campos se realiza por nombre.
- La generación automática de campos desde un adaptador CSV evita errores de tipeo.
- Los CSV pueden combinarse con bases de datos mediante subreportes o en memoria.
- La coherencia de nombres entre el CSV y el JRXML es condición necesaria.

Punto 3.3 — Ficheros XML

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.3 — Ficheros XML

Objetivos de aprendizaje

- Comprender el formato XML y su uso como fuente de datos estructurada.
- Configurar un adaptador XML en Jaspersoft Studio y asociarlo a un informe.
- Utilizar expresiones XPath para seleccionar registros y mapear campos.
- Leer un archivo XML desde código Java con `JRXmlDataSource`.
- Combinar datos jerárquicos de un XML con datos planos de un CSV.
- Documentar el uso de archivos XML en el proyecto EditorialReports.

Parte teórica

Bloque 1 — El formato XML como fuente de datos

XML (Extensible Markup Language) es un formato de texto plano que representa datos en una estructura jerárquica de elementos anidados. Cada elemento se delimita por una etiqueta de apertura y una etiqueta de cierre, y puede contener atributos, texto y otros elementos. Esta jerarquía permite representar relaciones complejas que un CSV no puede expresar: un libro puede tener varios autores, un pedido puede tener varias líneas, una categoría puede contener varias subcategorías. JasperReports incluye un módulo específico para leer archivos XML y convertirlos en fuentes de datos. A diferencia del CSV, el XML puede utilizarse directamente como estructura de datos sin una conversión previa a tabla.

XML

```
<catalogo>
  <libro disponible="true">
    <titulo>Cien años de soledad</titulo>
    <autor>Gabriel García Márquez</autor>
    <precio>19.95</precio>
    <paginas>471</paginas>
    <fecha_publicacion>1967-06-05</fecha_publicacion>
  </libro>
  <libro disponible="true">
    <titulo>Rayuela</titulo>
    <autor>Julio Cortázar</autor>
    <precio>22.50</precio>
    <paginas>736</paginas>
    <fecha_publicacion>1963-06-28</fecha_publicacion>
  </libro>
</catalogo>
```

Línea 1	<code><catalogo></code> → elemento raíz del documento. Contiene todos los elementos <code>libro</code> .
Línea 2	<code><libro disponible="true"></code> → elemento que representa un libro. El atributo <code>disponible</code> está declarado en el propio elemento, no como elemento hijo.
Línea 3	<code><titulo>Cien años de soledad</titulo></code> → elemento hijo con el título. El valor está contenido entre las etiquetas.
Línea 4	<code><autor>Gabriel García Márquez</autor></code> → elemento hijo con el autor.
Línea 5	<code><precio>19.95</precio></code> → elemento hijo con el precio.
Línea 6	<code><paginas>471</paginas></code> → elemento hijo con el número de páginas.
Línea 7	<code><fecha_publicacion>1967-06-05</fecha_publicacion></code> → elemento hijo con la fecha de publicación.
Línea 8	<code></libro></code> → cierre del elemento libro.
Línea 9-15	segundo elemento libro con la misma estructura.

El XML tiene dos características que lo diferencian del CSV. La primera es la jerarquía: los datos pueden anidarse en varios niveles, lo que permite representar relaciones uno-a-muchos sin duplicar información. La segunda es la presencia de atributos además de elementos: un valor puede almacenarse como texto dentro de una etiqueta o como atributo de la etiqueta. JasperReports permite acceder a ambos mediante expresiones XPath. La elección entre elemento y atributo depende del diseño del archivo y no afecta a la capacidad del motor para leerlo.

TEXT

DIFERENCIAS ENTRE CSV Y XML

Aspecto	CSV	XML
Estructura	Plana (filas y columnas)	Jerárquica (anidada)
Tipos de datos	Sin tipos (todo cadena)	Sin tipos (todo texto)
Relaciones	No soportadas	Soportadas por anidación
Atributos	No aplica	Soportados
Tamaño	Compacto	Verboso
Legibilidad humana	Alta	Alta
Validación	Sin esquema	Con XSD opcional
Uso típico	Intercambio simple	Configuración e integración

Qué representa la tabla: las diferencias entre CSV y XML en ocho aspectos. El XML es más expresivo pero más verboso.

Por qué es relevante: permite elegir el formato adecuado según la naturaleza de los datos. Los datos planos se representan mejor en CSV. Los datos con relaciones se representan mejor en XML.

Bloque 2 — Expresiones XPath en JasperReports

XPath es un lenguaje de consulta para seleccionar nodos dentro de un documento XML. JasperReports utiliza XPath para dos propósitos: seleccionar los elementos que actúan como registros y mapear cada campo a un nodo del registro. La selección de registros se realiza con una expresión XPath que devuelve una lista de nodos, como `//libro` o `/catalogo/libro`. El mapeo de campos se realiza con expresiones XPath relativas al nodo del registro, como `titulo` o `autor`. La combinación de ambas expresiones determina qué datos se extraen del XML y cómo se asignan a los campos del informe.

XML

```
<queryString language="XPath">
  <![CDATA[/catalogo/libro]]>
</queryString>
```

Línea 1	<code><queryString language="XPath"></code> → declara una consulta XPath. El atributo <code>language="XPath"</code> indica al motor que debe interpretar el contenido como una expresión XPath.
Línea 2	<code><![CDATA[/catalogo/libro]]></code> → expresión XPath que selecciona los elementos libro hijos de catalogo. Cada elemento seleccionado se convierte en un registro del informe.
Línea 3	<code></queryString></code> → cierra la declaración de la consulta.

Las expresiones XPath pueden incluir predicados que filtran los nodos según una condición. Un predicado se escribe entre corchetes y puede contener comparaciones con atributos o con valores de elementos. La expresión `//libro[@disponible='true']` selecciona los libros cuyo atributo `disponible` sea igual a `true`. La expresión `//libro[paginas > 200]` selecciona los libros cuyo elemento `paginas` sea mayor que 200. Los predicados permiten filtrar los registros sin necesidad de procesar el archivo completo en el programa Java. Esta característica es una de las ventajas del XML sobre el CSV.

XML

```
<queryString language="XPath">
  <![CDATA[/catalogo/libro[@disponible='true']]]>
</queryString>
```

Línea 2	<code><![CDATA[/catalogo/libro[@disponible='true']]]></code> → expresión XPath con predicado. Selecciona los elementos libro cuyo atributo <code>disponible</code> sea igual a la cadena <code>true</code> .
----------------	--

TEXT

EXPRESIONES XPATH FRECUENTES

Selección de registros:

- `/catalogo/libro` → todos los libros hijos de catalogo
- `//libro` → todos los elementos libro del documento
- `/catalogo/libro[@disponible='true']` → libros disponibles
- `/catalogo/libro[paginas > 200]` → libros con más de 200 páginas

Mapeo de campos (relativo al nodo del registro):

- `titulo` → elemento titulo
- `autor` → elemento autor
- `@disponible` → atributo disponible
- `precio` → elemento precio
- `fecha_publicacion` → elemento fecha_publicacion

Funciones:

- count(libro) → número de libros
- string-length(titulo) → longitud del título
- concat(autor, '(', titulo, ')') → concatenación

Qué representa el diagrama: las expresiones XPath más utilizadas en JasperReports, agrupadas por función. Las de selección determinan los registros. Las de mapeo determinan los campos. Las funciones permiten cálculos.

Por qué es relevante: permite escribir las expresiones necesarias para leer cualquier archivo XML sin recurrir a la documentación completa de XPath.

Bloque 3 — Configuración del adaptador XML en Jaspersoft Studio

Jaspersoft Studio permite configurar adaptadores XML desde el panel Repository Explorer con la opción XML File Data Source. En el asistente se especifica la ruta del archivo y la expresión XPath que selecciona los registros. El entorno lee el archivo y ejecuta la expresión para identificar los nodos que actuarán como registros. Una vez configurado, el adaptador se asocia al informe mediante la propiedad `com.jaspersoft.studio.data.defaultdataadapter`. La asociación permite que el botón Preview ejecute el informe con los datos del XML.

TEXT

```
Repository Explorer > Data Adapters > Create Data Adapter
→ XML File Data Source
→ Name: DistribucionXML
→ File: data/distribucion.xml
→ XPath: /distribucion/entrega
→ Test Connection → OK
```

Línea 1	Repository Explorer > Data Adapters > Create Data Adapter → ruta de menú para crear un nuevo adaptador.
Línea 2	XML File Data Source → tipo de adaptador XML.
Línea 3	Name: DistribucionXML → nombre del adaptador en el panel Repository Explorer.
Línea 4	File: data/distribucion.xml → ruta del archivo XML. La ruta es relativa al directorio de trabajo del entorno.
Línea 5	XPath: /distribucion/entrega → expresión XPath que selecciona los elementos entrega como registros.
Línea 6	Test Connection → OK → botón que verifica que el archivo se lee y la expresión XPath se ejecuta.

El adaptador XML construye una colección de nodos DOM que el motor recorre uno a uno. Cada nodo representa un registro y el motor resuelve los campos evaluando las expresiones XPath relativas a ese nodo. La resolución se realiza en el momento de la emisión de la banda que contiene la expresión. Si una expresión XPath no devuelve ningún nodo, el campo se resuelve como null. La coherencia entre las expresiones XPath del JRXML y la estructura del archivo XML es condición necesaria para que los campos se resuelvan.

TEXT

LECTURA DEL XML POR EL ADAPTADOR

```
Archivo XML:
<distribucion>
  <entrega>
    <libreria>Librería Central</libreria>
    <ciudad>Madrid</ciudad>
    <cantidad>25</cantidad>
  </entrega>
</distribucion>

Expresión XPath de selección: /distribucion/entrega
Nodos seleccionados: 1 (el elemento entrega)

El motor resuelve $F{libreria} evaluando la expresión XPath
"libreria" relativa al nodo del registro actual.
```

Qué representa el diagrama: la conversión del archivo XML en una colección de nodos que el motor recorre. La expresión XPath de selección determina los registros. Las expresiones XPath de los campos se evalúan relativas al nodo del registro.

Por qué es relevante: permite comprender que la expresión XPath de selección es diferente de las expresiones XPath de los campos. La primera es absoluta, la segunda es relativa.

Bloque 4 — Lectura de XML desde código Java

La lectura de un archivo XML desde código Java se realiza con la clase `net.sf.jasperreports.engine.data.JRXmlDataSource`. Esta clase lee el archivo, ejecuta la expresión XPath de selección y construye una fuente de datos que el motor puede recorrer. El

constructor recibe la ruta del archivo y la expresión XPath. La expresión XPath que selecciona los registros se pasa al constructor. El mapeo de cada campo se declara con la propiedad `net.sf.jasperreports.xpath.field.expression`; `JRXmlDataSource` no utiliza `setFieldDelimiter`, que pertenece a las fuentes CSV. La fuente de datos se pasa al motor de llenado como tercer argumento de `fillReport`.

JAVA

```
JRXmlDataSource dataSource = new JRXmlDataSource("data/distribucion.xml", "/distribucion/entrega");
```

Línea 1

`JRXmlDataSource dataSource = new JRXmlDataSource("data/distribucion.xml", "/distribucion/entrega");` → construye una fuente de datos XML a partir del archivo indicado y la expresión XPath que selecciona los registros.

La clase `JRXmlDataSource` carga el archivo completo en memoria y construye un árbol DOM. Esta característica permite evaluar expresiones XPath con predicados y funciones, pero implica que el archivo entero se mantiene en memoria durante el llenado. Para archivos muy grandes, esta característica puede consumir una cantidad significativa de memoria. La contrapartida es la flexibilidad: el XML puede representar relaciones complejas sin necesidad de convertir el archivo a otra estructura. La clase forma parte de la biblioteca JasperReports y no requiere dependencias adicionales.

TEXT

FLUJO DE LECTURA DEL XML

1. `JRXmlDataSource` carga el archivo XML en memoria.
2. El árbol DOM se construye completo.
3. La expresión XPath de selección se evalúa contra el árbol.
4. Los nodos seleccionados forman la lista de registros.
5. El motor recorre los nodos uno a uno.
6. Para cada nodo, se evalúan las expresiones XPath de los campos.
7. El resultado se imprime en el documento.

Qué representa el diagrama: el flujo de lectura de un archivo XML con `JRXmlDataSource`. El archivo se carga completo en memoria antes de recorrerlo.

Por qué es relevante: permite comprender el consumo de memoria y el comportamiento de la fuente de datos durante el llenado.

Bloque 5 — Combinación de XML con otras fuentes

El XML es especialmente útil para representar datos jerárquicos que se combinan con otras fuentes. Un caso típico es un informe de distribución que combina datos del catálogo (que residen en un CSV) con datos de las entregas a librerías (que residen en un XML). La combinación puede realizarse de dos formas. La primera es mediante subreportes: el informe principal se alimenta del CSV y cada registro invoca un subreporte que se alimenta del XML. La segunda es mediante la construcción de una fuente de datos combinada en memoria: el programa Java lee ambas fuentes, construye una lista de objetos que contiene los datos combinados y la pasa al motor. La elección entre ambas depende del volumen de datos y de la complejidad de la combinación.

TEXT

COMBINACIÓN DE CSV Y XML

```
Informe principal (alimentado por CSV):
├── Título del libro (columna del CSV)
├── Precio (columna del CSV)
└── Subreporte (alimentado por XML):
    └── Distribución a librerías (elementos entrega del XML)
```

El subreporte recibe el título del libro como parámetro.

Qué representa el diagrama: la estructura de un informe que combina un CSV en el nivel principal con un XML en un subreporte. El subreporte recibe un parámetro del nivel principal.

Por qué es relevante: permite ampliar el alcance del informe sin renunciar a la expresividad del XML. Los datos planos residen en el CSV y los datos jerárquicos en el XML.

La combinación también puede realizarse dentro del mismo nivel del informe. Un informe puede leer un archivo XML que contiene toda la información necesaria, incluyendo datos que en otro diseño residirían en varias fuentes. El XML permite anidar la información relacionada en un único archivo. Un catálogo completo puede representarse como un XML que contiene los libros, los autores y las categorías en una estructura jerárquica. El motor lee el archivo una sola vez y resuelve todas las expresiones XPath necesarias. Esta aproximación simplifica la gestión de fuentes de datos a cambio de un archivo XML más complejo.

XML

```
<catalogo>
  <libro disponible="true">
    <titulo>Cien años de soledad</titulo>
    <autores>
      <autor>Gabriel García Márquez</autor>
    </autores>
    <categorías>
      <categoria>Realismo mágico</categoria>
      <categoria>Novela</categoria>
    </categorías>
  </libro>
</catalogo>
```

```
</categorias>  
</libro>  
</catalogo>
```

Línea 1-9

estructura jerárquica con elementos anidados. El elemento `libro` contiene un elemento `autores` que a su vez contiene elementos `autor`. La expresión XPath `autores/autor` selecciona los autores. La expresión `categorias/categoria` selecciona las categorías.

Resumen rápido de la teoría

- XML es un formato jerárquico que representa datos en elementos anidados.
- JasperReports utiliza XPath para seleccionar registros y mapear campos.
- La expresión XPath de selección es absoluta. Las de campo son relativas al nodo del registro.
- El adaptador XML de Jaspersoft Studio se configura desde el Repository Explorer.
- La clase `JRXm1DataSource` permite leer XML desde código Java.
- Los predicados XPath permiten filtrar los registros.
- El XML se carga completo en memoria durante el llenado.
- El XML puede combinarse con otras fuentes mediante subreportes.

Punto 3.4 — Ficheros JSON

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.4 — Ficheros JSON

Objetivos de aprendizaje

- Comprender el formato JSON y su uso como fuente de datos estructurada.
- Configurar un adaptador JSON en Jaspersoft Studio y asociarlo a un informe.
- Utilizar expresiones JSON para seleccionar registros y mapear campos.
- Leer un archivo JSON desde código Java con `JsonDataSource`.
- Combinar datos de un JSON con datos de otras fuentes del proyecto.
- Documentar el uso de archivos JSON en el proyecto EditorialReports.

Parte teórica

Bloque 1 — El formato JSON como fuente de datos

JSON (JavaScript Object Notation) es un formato de texto plano que representa datos estructurados mediante pares de clave-valor y colecciones ordenadas. Su sintaxis se compone de objetos delimitados por llaves, arreglos delimitados por corchetes y valores que pueden ser cadenas, números, booleanos, nulos, objetos o arreglos. Esta estructura resulta especialmente adecuada para representar datos que provienen de servicios web y de APIs REST, que es el origen más frecuente de los archivos JSON en el ámbito empresarial. En el proyecto EditorialReports, el archivo JSON contiene la información de los autores de los libros, con datos biográficos y de contacto que no residen en la base de datos ni en el CSV del catálogo.

JSON

```
{
  "autores": [
    {
      "nombre": "Gabriel García Márquez",
      "nacionalidad": "Colombiana",
      "nacimiento": "1927-03-06",
      "premios": 1,
      "vivo": false
    },
    {
      "nombre": "Julio Cortázar",
      "nacionalidad": "Argentina",
      "nacimiento": "1914-08-26",
      "premios": 0,
      "vivo": false
    }
  ]
}
```

Línea 1-2	{ "autores": [→ abre el objeto raíz y declara el arreglo autores que contiene los objetos de cada autor.
Línea 3-9	primer objeto de autor con cinco propiedades: nombre, nacionalidad, nacimiento, premios y vivo. Cada propiedad es un par clave-valor separado por dos puntos.
Línea 10-16	segundo objeto de autor con la misma estructura.
Línea 17-18	] } → cierra el arreglo y el objeto raíz.

A diferencia del XML, JSON no utiliza etiquetas con nombre repetido para los elementos de un arreglo. En su lugar, utiliza una estructura de arreglo delimitada por corchetes que contiene objetos anónimos. Este diseño resulta más compacto y más legible que el XML para datos tabulares. La contrapartida es que la estructura jerárquica de JSON es menos flexible que la de XML para representar documentos con múltiples niveles de anidación. En la práctica, JSON se utiliza para datos con uno o dos niveles de anidación y XML para documentos con estructuras más profundas. La elección entre ambos depende de la naturaleza del origen de datos.

DIFERENCIAS ENTRE XML Y JSON

TEXT

Aspecto	XML	JSON
---------	-----	------

Sintaxis	Etiquetas anidadas	Llaves y corchetes
Verbosidad	Alta	Baja
Legibilidad humana	Alta	Alta
Tipos de datos	Solo texto	Texto, número, booleano, nulo, objeto, arreglo
Atributos	Soportados	No aplica (todo son claves)
Validación	XSD	JSON Schema
Origen típico	Configuración, documentos	APIs REST, servicios web
Anidación	Profunda	Moderada

Qué representa la tabla: las diferencias entre XML y JSON en ocho aspectos. JSON es más compacto y soporta tipos de datos. XML es más expresivo para estructuras profundas.

Por qué es relevante: permite elegir el formato adecuado según la naturaleza del origen de datos. Los servicios web modernos suelen devolver JSON. Los documentos de configuración suelen utilizar XML.

Bloque 2 — Expresiones JSON en JasperReports

JasperReports 6.20.0 incluye un módulo específico para leer archivos JSON denominado `j son` que se activa cuando el archivo JAR `jackson-*.jar` está en el classpath. El lenguaje JSON clásico de JasperReports 6.20.0 usa navegación por propiedades con punto y filtros entre paréntesis; no es la sintaxis `JSONPath $[?()]` habitual de otras bibliotecas. La expresión de selección de registros se declara en el atributo `language` del elemento `queryString` con el valor `json`. Las expresiones de los campos utilizan la notación de puntos para acceder a las propiedades anidadas. La sintaxis es más simple que la de XPath pero también menos potente para consultas complejas.

XML

```
<queryString language="json">
  <![CDATA[autores]]>
</queryString>
```

Línea 1

`<queryString language="j son">` → declara una consulta JSON. El atributo `language="j son"` indica al motor que debe interpretar el contenido como una expresión de selección JSON.

Línea 2

`<![CDATA[autores]]>` → expresión que selecciona el arreglo `autores` del objeto raíz. Cada elemento del arreglo se convierte en un registro del informe.

Línea 3

`</queryString>` → cierra la declaración de la consulta.

Las expresiones de los campos se declaran en el atributo `name` de los elementos `field` y utilizan la notación de puntos para acceder a propiedades anidadas. Un campo declarado como `nombre` accede a la propiedad `nombre` del objeto actual. Un campo declarado como `direccion.ciudad` accede a la propiedad `ciudad` dentro de la propiedad `direccion` del objeto actual. El lenguaje JSON clásico de JasperReports permite filtros entre paréntesis, por ejemplo `autores(premios > 0)`. No debe confundirse con la sintaxis `JSONPath` externa `$[?()]`. El filtro se evalúa al construir la fuente de datos y permite seleccionar un subconjunto de registros.

XML

```
<queryString language="json">
  <![CDATA[autores(premios > 0)]]>
</queryString>
```

Línea 2

`<![CDATA[autores(premios > 0)]]>` → expresión con filtro. Selecciona los autores cuyo campo `premios` sea superior a cero. Los autores sin premios quedan excluidos del informe.

TEXT

EXPRESIONES JSON FRECUENTES

Selección de registros:

```
autores                → todos los autores
autores(premios > 0)    → autores con premios
autores(nacionalidad == Argentina) → autores argentinos
libros[*]              → todos los elementos del arreglo libros
```

Mapeo de campos (relativo al registro):

```
nombre                → propiedad nombre
nacionalidad          → propiedad nacionalidad
nacimiento            → propiedad nacimiento
premios               → propiedad premios
direccion.ciudad      → propiedad anidada
```

Tipos de datos:

Las propiedades se convierten automáticamente al tipo declarado en el campo del JRXML: `String`, `Integer`, `Boolean`, `Date`.

Qué representa el diagrama: las expresiones JSON más utilizadas en JasperReports, agrupadas por función. Las de selección determinan los registros. Las de mapeo determinan los campos.

Por qué es relevante: permite escribir las expresiones necesarias para leer un archivo JSON sin confundir este lenguaje con JSONPath de otras bibliotecas.

Bloque 3 — Configuración del adaptador JSON en Jaspersoft Studio

Jaspersoft Studio permite configurar adaptadores JSON desde el panel Repository Explorer con la opción JSON File Data Source. En el asistente se especifica la ruta del archivo y la expresión de selección de registros. El entorno lee el archivo y ejecuta la expresión para identificar los nodos que actuarán como registros. Una vez configurado, el adaptador se asocia al informe mediante la propiedad `com.jaspersoft.studio.data.defaultdataadapter`. La asociación permite que el botón Preview ejecute el informe con los datos del JSON.

TEXT

```
Repository Explorer > Data Adapters > Create Data Adapter
→ JSON File Data Source
→ Name: AutoresJSON
→ File: data/autores.json
→ Select Expression: autores
→ Test Connection → OK
```

Línea 1	Repository Explorer > Data Adapters > Create Data Adapter → ruta de menú para crear un nuevo adaptador.
Línea 2	JSON File Data Source → tipo de adaptador JSON.
Línea 3	Name: AutoresJSON → nombre del adaptador en el panel Repository Explorer.
Línea 4	File: data/autores.json → ruta del archivo JSON.
Línea 5	Select Expression: autores → expresión que selecciona el arreglo de autores como registros.
Línea 6	Test Connection → OK → botón que verifica que el archivo se lee y la expresión se ejecuta.

El adaptador JSON convierte los valores del archivo al tipo declarado en el campo del JRXML. Un campo declarado como `java.lang.Integer` recibe un número entero. Un campo declarado como `java.lang.Boolean` recibe un valor booleano. Los números y booleanos pueden convertirse directamente a `Integer`, `Double` o `Boolean`. Las fechas JSON siguen siendo cadenas salvo que se configure explícitamente un patrón de fecha. En EditorialReports nacimiento se mantiene como `String ISO` para que la práctica sea determinista. La contrapartida es que el tipo declarado debe coincidir con el tipo del valor del JSON, y una discrepancia produce un error de conversión.

TEXT

CONVERSIÓN AUTOMÁTICA DE TIPOS EN JSON

Archivo JSON:

```
{ "premios": 1, "vivo": false, "nacimiento": "1927-03-06" }
```

Campos del JRXML:

```
<field name="premios" class="java.lang.Integer"/> → recibe 1
<field name="vivo" class="java.lang.Boolean"/> → recibe false
<field name="nacimiento" class="java.lang.String"/> → recibe la fecha ISO como texto
```

El motor convierte los tipos numéricos y booleanos; la fecha se conserva como texto en esta práctica. Si el tipo declarado no coincide con el tipo del valor, el motor lanza una excepción de conversión.

Qué representa el diagrama: la conversión automática de tipos que realiza el adaptador JSON. Los valores del archivo se convierten al tipo declarado en el campo del JRXML.

Por qué es relevante: permite declarar los campos con el tipo final y evitar conversiones manuales en las expresiones. Es una ventaja frente a CSV y XML.

Bloque 4 — Lectura de JSON desde código Java

La lectura de un archivo JSON desde código Java se realiza con la clase `net.sf.jasperreports.engine.data.JsonDataSource`. Esta clase lee el documento y construye una fuente de datos que el motor puede recorrer. Sus constructores admiten, entre otras opciones, un `File` o un `InputStream` junto con la expresión de selección. La fuente de datos se pasa al motor de llenado como tercer argumento de `fillReport`. El soporte JSON utiliza Jackson, cuyas clases deben estar disponibles en el classpath; en el runtime reproducible del curso Maven las resuelve transitivamente.

JAVA

```
JsonDataSource dataSource = new JsonDataSource(new File("data/autores.json"), "autores");
```

Línea 1

`JsonDataSource dataSource = new JsonDataSource(new File("data/autores.json"), "autores");` → abre el archivo JSON y aplica explícitamente la selección `autores`. Cuando se pasa este `JRDataSource` a `fillReport`, la consulta JSON del JRXML no se ejecuta de nuevo.

La clase `JsonDataSource` lee el archivo completo en memoria y construye un árbol de objetos que el motor recorre. Esta característica es similar a la de `JRXmlDataSource` y permite evaluar expresiones con filtros. La contrapartida es el consumo de memoria para archivos grandes. La implementación JSON utiliza Jackson. En la ejecución reproducible del curso, Maven resuelve las dependencias transitivas necesarias desde `jasperreports:6.20.0`; no es necesario declarar tres dependencias Jackson adicionales en el `pom.xml`. En el ejercicio manual dentro de Jaspersoft Studio, si el proyecto Java se gestiona únicamente mediante Build Path, los JAR de Jackson pueden añadirse desde la distribución instalada, tal como se documenta en la práctica.

TEXT

FLUJO DE LECTURA DEL JSON

1. `JsonDataSource` lee el archivo JSON.
2. El módulo Jackson construye el árbol de objetos.
3. La expresión de selección del JRXML se evalúa contra el árbol.
4. Los elementos seleccionados forman la lista de registros.
5. El motor recorre los registros uno a uno.
6. Para cada registro, se evalúan las expresiones de los campos.
7. El resultado se imprime en el documento.

Qué representa el diagrama: el flujo de lectura de un archivo JSON con `JsonDataSource`. El archivo se carga completo en memoria antes de recorrerlo.

Por qué es relevante: permite comprender el consumo de memoria y la dependencia del módulo Jackson para la lectura del JSON.

Bloque 5 — Combinación de JSON con otras fuentes

El JSON es especialmente útil para representar datos que provienen de servicios web o de APIs REST. Un caso típico es un informe que combina datos del catálogo (que residen en una base de datos) con datos de autores obtenidos de una API en formato JSON. La combinación puede realizarse de dos formas. La primera es mediante subreportes: el informe principal se alimenta de la base de datos y cada registro invoca un subreporte que se alimenta del JSON. La segunda es mediante la construcción de una fuente de datos combinada en memoria: el programa Java lee ambas fuentes, construye una lista de objetos que contiene los datos combinados y la pasa al motor. La elección entre ambas depende del volumen de datos y de la complejidad de la combinación.

TEXT

COMBINACIÓN DE BASE DE DATOS Y JSON

```
Informe principal (alimentado por JDBC):
├── Título del libro (columna de la base de datos)
├── Precio (columna de la base de datos)
└── Subreporte (alimentado por JSON):
    └── Datos del autor (objetos del JSON)
```

El subreporte recibe el nombre del autor como parámetro.

Qué representa el diagrama: la estructura de un informe que combina una base de datos en el nivel principal con un JSON en un subreporte. El subreporte recibe un parámetro del nivel principal.

Por qué es relevante: permite ampliar el alcance del informe sin renunciar a la expresividad del JSON. Los datos estructurados residen en la base de datos y los datos de servicios externos en el JSON.

La combinación también puede realizarse dentro del mismo nivel del informe. Un informe puede leer un archivo JSON que contiene toda la información necesaria, incluyendo datos que en otro diseño residirían en varias fuentes. El JSON permite anidar la información relacionada en un único archivo. Un catálogo completo puede representarse como un JSON que contiene los libros, los autores y las categorías en una estructura jerárquica. El motor lee el archivo una sola vez y resuelve todas las expresiones necesarias. Esta aproximación simplifica la gestión de fuentes de datos a cambio de un archivo JSON más complejo.

JSON

```
{
  "libros": [
    {
      "titulo": "Cien años de soledad",
      "precio": 19.95,
      "autor": {
        "nombre": "Gabriel García Márquez",
        "nacionalidad": "Colombiana"
      },
      "categorias": ["Realismo mágico", "Novela"]
    }
  ]
}
```

Línea 1-2

`{ "libros": [` → abre el objeto raíz y declara el arreglo de libros.

Línea 3-10	primer objeto de libro con las propiedades <code>titulo</code> , <code>precio</code> , <code>autor</code> y <code>categorias</code> . La propiedad <code>autor</code> es un objeto anidado. La propiedad <code>categorias</code> es un arreglo de cadenas.
Línea 4	<code>"titulo": "Cien años de soledad"</code> → propiedad simple.
Línea 6-9	objeto anidado <code>autor</code> con dos propiedades.
Línea 10	arreglo <code>categorias</code> con dos cadenas.

Resumen rápido de la teoría

- JSON es un formato de texto plano que representa datos mediante pares clave-valor y arreglos.
 - JasperReports 6.20.0 usa su lenguaje JSON clásico: navegación por puntos y filtros entre paréntesis; no es `JSONPath $[?()]`.
 - El adaptador JSON de Jaspersoft Studio se configura desde el Repository Explorer.
 - La clase `JsonDataSource` permite leer JSON desde código Java.
 - JSON convierte automáticamente los tipos de datos según el tipo declarado en el campo.
 - El soporte JSON utiliza Jackson; Maven resuelve sus dependencias transitivas en la ejecución reproducible y el Build Path manual debe contenerlas cuando se trabaja fuera de Maven.
 - Los filtros JSON permiten seleccionar subconjuntos de registros.
 - El JSON puede combinarse con otras fuentes mediante subreportes.
-
-

Punto 3.5 — Consultas SQL

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.5 — Consultas SQL

Objetivos de aprendizaje

- Escribir consultas SQL completas dentro del elemento `queryString` del JRXML.
- Utilizar alias de columna para mapear nombres de la base de datos a nombres de campo.
- Aplicar funciones de agregación (SUM, AVG, COUNT, MAX, MIN) en las consultas.
- Construir consultas con JOIN entre dos o más tablas.
- Utilizar GROUP BY y ORDER BY para agrupar y ordenar resultados.
- Documentar las consultas SQL del proyecto EditorialReports.

Parte teórica

Bloque 1 — Sintaxis básica de las consultas SQL

Una consulta SQL declarada en el JRXML se escribe dentro del elemento `queryString` con el atributo `language="sql"`. La consulta se encierra en un bloque CDATA para evitar conflictos con los caracteres especiales del XML, como el signo mayor > o la comilla simple. La sintaxis es la del lenguaje SQL estándar, con las extensiones específicas del motor de base de datos. En el proyecto EditorialReports se utiliza SQLite, que soporta la mayor parte del estándar y añade algunas funciones específicas. La consulta devuelve un conjunto de filas y columnas que el motor mapea a los campos del informe por nombre.

XML

```
<queryString language="sql">
  <![CDATA[
    SELECT titulo, precio, paginas
    FROM libros
    WHERE disponible = 1
    ORDER BY precio DESC
  ]]>
</queryString>
```

Línea 1	<code><queryString language="sql"></code> → declara una consulta SQL. El atributo <code>language="sql"</code> indica al motor que debe interpretar el contenido como una sentencia SQL.
Línea 2	<code><![CDATA[</code> → abre el bloque CDATA que protege el contenido de caracteres especiales.
Línea 3	<code>SELECT titulo, precio, paginas</code> → indica las columnas que se recuperan de la tabla. El resultado tendrá tres columnas.
Línea 4	<code>FROM libros</code> → indica la tabla de origen.
Línea 5	<code>WHERE disponible = 1</code> → filtra las filas. Solo se recuperan los libros con <code>disponible = 1</code> (SQLite no tiene tipo booleano nativo y utiliza 1 para verdadero y 0 para falso).
Línea 6	<code>ORDER BY precio DESC</code> → ordena las filas por precio en orden descendente.
Línea 7	<code>]]></code> → cierra el bloque CDATA.
Línea 8	<code></queryString></code> → cierra la declaración de la consulta.

La correspondencia entre las columnas del `ResultSet` y los campos del JRXML se establece por nombre. El motor toma cada columna del resultado y la mapea al campo cuyo nombre coincide. Si una columna del `ResultSet` no tiene un campo correspondiente en el JRXML, se ignora. Si un campo del JRXML no tiene una columna correspondiente en el `ResultSet`, el motor lanza `Field not found` al resolver la expresión que lo referencia. La coherencia entre los nombres de las columnas y los nombres de los campos es condición necesaria para que la resolución funcione. Esta coherencia puede conseguirse de dos formas: nombrando las columnas de la consulta con los mismos nombres que los campos del JRXML, o utilizando alias de columna.

TEXT

CORRESPONDENCIA ENTRE COLUMNAS Y CAMPOS

```

Consulta SQL:
SELECT titulo, precio, paginas
FROM libros

Campos del JRXML:
<field name="titulo" class="java.lang.String"/>
<field name="precio" class="java.lang.Double"/>
<field name="paginas" class="java.lang.Integer"/>

El motor mapea:
columna "titulo" → ${F{titulo}}
columna "precio" → ${F{precio}}
columna "paginas" → ${F{paginas}}

```

Qué representa el diagrama: la correspondencia por nombre entre las columnas del `ResultSet` y los campos del JRXML. Los nombres deben coincidir exactamente.

Por qué es relevante: permite diagnosticar errores de resolución de campos cuando se trabaja con consultas SQL. Si un campo no se resuelve, la causa suele ser una discrepancia entre el nombre de la columna y el nombre del campo.

Bloque 2 — Alias de columna y expresiones calculadas

Un alias de columna es un nombre alternativo que se asigna a una columna del `ResultSet` con la cláusula `AS`. El alias sustituye al nombre original de la columna en el resultado y es el nombre que el motor utiliza para mapear al campo del JRXML. Los alias permiten dos cosas: renombrar columnas cuyo nombre original no coincide con el nombre deseado y nombrar columnas calculadas que no existen en la tabla. Una columna calculada es el resultado de una operación aritmética o de una función aplicada a una o varias columnas. El alias es obligatorio para las columnas calculadas porque no tienen un nombre original.

XML

```

<queryString language="sql">
  <![CDATA[
    SELECT titulo,
           precio * 1.21 AS precio_con_iva,
           paginas / 10 AS decenas_paginas
    FROM libros
    ORDER BY titulo
  ]]>
</queryString>

```

Línea 3	<code>SELECT titulo,</code> → recupera la columna <code>titulo</code> sin alias. El campo del JRXML debe llamarse <code>titulo</code> .
Línea 4	<code>precio * 1.21 AS precio_con_iva,</code> → calcula el precio con IVA y le asigna el alias <code>precio_con_iva</code> . El campo del JRXML debe llamarse <code>precio_con_iva</code> .
Línea 5	<code>paginas / 10 AS decenas_paginas</code> → calcula las decenas de páginas y le asigna el alias <code>decenas_paginas</code> .
Línea 6	<code>FROM libros</code> → indica la tabla de origen.
Línea 7	<code>ORDER BY titulo</code> → ordena las filas por título en orden ascendente.

Los alias permiten también construir consultas con nombres de columna compuestos o con caracteres que no son válidos en un nombre de campo de JasperReports. Un alias puede contener guiones bajos, números y letras. Los alias que contienen espacios o caracteres especiales deben encerrarse entre comillas dobles en SQL, pero esta práctica no se recomienda porque complica la legibilidad. La convención en el proyecto EditorialReports es usar alias en minúsculas con guiones bajos para separar palabras. Esta convención coincide con la de los nombres de campo y evita conversiones innecesarias.

TEXT

COLUMNAS CALCULADAS CON ALIAS

```

Consulta SQL:
SELECT titulo,
       precio * 1.21 AS precio_iva,
       paginas / 10 AS decenas,
       precio + 5 AS precio_mas_5
FROM libros

El motor calcula las tres columnas calculadas para cada fila.
Los campos del JRXML se declaran con los alias:
<field name="titulo" class="java.lang.String"/>
<field name="precio_iva" class="java.lang.Double"/>
<field name="decenas" class="java.lang.Integer"/>
<field name="precio_mas_5" class="java.lang.Double"/>

```

Qué representa el diagrama: la creación de columnas calculadas con alias en la consulta SQL. Los campos del JRXML toman los nombres de los alias.

Por qué es relevante: permite construir valores derivados desde la propia consulta SQL sin necesidad de calcularlos en expresiones del JRXML. Este enfoque simplifica la plantilla y concentra los cálculos en la capa de datos.

Bloque 3 — Parámetros en las consultas SQL

Una consulta SQL puede incluir parámetros que se enlazan en el momento del llenado. Con `$P{nombre}` JasperReports prepara la sentencia JDBC y utiliza un marcador `?`; el valor se asigna después con el tipo Java del parámetro. No se concatena literalmente dentro del SQL. Para sustitución textual existe `$P!{nombre}`, que debe reservarse para fragmentos controlados porque no ofrece las mismas garantías de parametrización. Los parámetros se estudian con detalle en el Módulo 4; en este punto se introduce su sintaxis para que las consultas del informe sean completas.

XML

```
<parameter name="precioMinimo" class="java.lang.Double"/>
<queryString language="sql">
  <![CDATA[
    SELECT titulo, precio, paginas
    FROM libros
    WHERE precio >= $P{precioMinimo}
    ORDER BY precio DESC
  ]]>
</queryString>
```

Línea 1	<code><parameter name="precioMinimo" class="java.lang.Double"/></code> → declara un parámetro de tipo Double. El valor se pasa desde el programa Java en el mapa de parámetros.
Línea 3	<code><![CDATA[</code> → abre el bloque CDATA.
Línea 4	<code>SELECT titulo, precio, paginas</code> → indica las columnas.
Línea 5	<code>FROM libros</code> → indica la tabla.
Línea 6	<code>WHERE precio >= \$P{precioMinimo}</code> → filtra las filas. JasperReports transforma <code>\$P{precioMinimo}</code> en un marcador JDBC y enlaza después el valor tipado.
Línea 7	<code>ORDER BY precio DESC</code> → ordena las filas por precio descendente.

Los parámetros SQL admiten varios tipos Java. Con `$P{}` no se construye el literal concatenando comillas: JasperReports usa una sentencia preparada y asigna cada valor con el tipo correspondiente. Un `String`, un número o una fecha se envían al driver como parámetros JDBC. El tipo declarado determina cómo se enlaza el valor. Un parámetro mal tipificado puede producir errores de sintaxis SQL o resultados incorrectos. La coherencia entre el tipo del parámetro y el tipo de la columna con la que se compara es condición necesaria para que el filtro funcione.

TEXT

SUSTITUCIÓN DE PARÁMETROS EN LA CONSULTA

```
Parámetro declarado:
<parameter name="tituloBuscado" class="java.lang.String"/>

Consulta declarada:
SELECT titulo, precio FROM libros WHERE titulo = $P{tituloBuscado}

Valor del parámetro en el programa Java:
parametros.put("tituloBuscado", "Rayuela");

Forma preparada equivalente:
SELECT titulo, precio FROM libros WHERE titulo = ?

Valor enlazado al marcador: Rayuela

El motor añade las comillas simples alrededor del valor
porque el parámetro es de tipo String.
```

Qué representa el diagrama: el enlace tipado de un parámetro `String` mediante JDBC. No hay concatenación manual del literal en el SQL.

Por qué es relevante: permite escribir consultas parametrizadas sin preocuparse por el escape de las comillas. El motor gestiona la conversión según el tipo declarado.

Bloque 4 — Funciones de agregación

Las funciones de agregación permiten calcular valores a partir de un conjunto de filas. Las funciones estándar son `SUM`, `AVG`, `COUNT`, `MAX` y `MIN`. La función `SUM` calcula la suma de una columna numérica. La función `AVG` calcula la media. La función `COUNT` cuenta el número de filas. La función `MAX` devuelve el valor máximo. La función `MIN` devuelve el valor mínimo. Cuando una consulta utiliza funciones de agregación, el resultado tiene una sola fila con los valores calculados, a menos que se utilice la cláusula `GROUP BY` para agrupar los resultados por una o varias columnas.

XML

```
<queryString language="sql">
  <![CDATA[
```

```

SELECT COUNT(*) AS total_libros,
       SUM(precio) AS suma_precios,
       AVG(precio) AS precio_medio,
       MAX(precio) AS precio_maximo,
       MIN(precio) AS precio_minimo
FROM libros
]]>
</queryString>

```

Línea 3	SELECT COUNT(*) AS total_libros, → cuenta el número total de filas de la tabla y le asigna el alias total_libros.
Línea 4	SUM(precio) AS suma_precios, → suma todos los precios y le asigna el alias suma_precios.
Línea 5	AVG(precio) AS precio_medio, → calcula la media de los precios.
Línea 6	MAX(precio) AS precio_maximo, → calcula el precio máximo.
Línea 7	MIN(precio) AS precio_minimo → calcula el precio mínimo.
Línea 8	FROM libros → indica la tabla.

La cláusula **GROUP BY** agrupa las filas por el valor de una o varias columnas y aplica las funciones de agregación a cada grupo. El resultado tiene una fila por cada valor distinto de la columna de agrupación. La cláusula **ORDER BY** puede aplicarse después del **GROUP BY** para ordenar los grupos. La combinación de **GROUP BY** y funciones de agregación permite construir informes con subtotales por categoría, por año o por cualquier otro criterio. La cláusula **HAVING** permite filtrar los grupos después de la agregación, de forma similar a como **WHERE** filtra las filas antes de la agregación.

```

<queryString language="sql">
  <![CDATA[
    SELECT CASE WHEN disponible = 1 THEN 'Disponible' ELSE 'No disponible' END AS estado,
           COUNT(*) AS num_libros,
           AVG(precio) AS precio_medio
    FROM libros
    GROUP BY disponible
    HAVING COUNT(*) > 1
    ORDER BY num_libros DESC
  ]]>
</queryString>

```

XML

Línea 3	la expresión CASE ... END AS estado transforma el 1/0 almacenado en SQLite en una etiqueta legible.
Línea 4	COUNT(*) AS num_libros cuenta los libros de cada grupo.
Línea 5	AVG(precio) AS precio_medio calcula la media de precios del grupo.
Línea 6	FROM libros usa la tabla real del proyecto.
Línea 7	GROUP BY disponible crea un grupo para disponibles y otro para no disponibles.
Línea 8	HAVING COUNT(*) > 1 conserva los grupos con más de un libro.
Línea 9	ORDER BY num_libros DESC ordena por tamaño de grupo.

Bloque 5 — Consultas con JOIN

La cláusula **JOIN** permite combinar filas de dos o más tablas según una condición de coincidencia. El **INNER JOIN** devuelve solo las filas que tienen coincidencia en ambas tablas. El **LEFT JOIN** devuelve todas las filas de la tabla izquierda y las coincidentes de la derecha, con valores nulos cuando no hay coincidencia. El **RIGHT JOIN** funciona al revés. El **FULL OUTER JOIN** devuelve todas las filas de ambas tablas, aunque no tengan coincidencia. La elección del tipo de **JOIN** determina qué filas aparecen en el resultado.

```

<queryString language="sql">
  <![CDATA[
    SELECT l.titulo,
           l.precio,
           v.cantidad,
           v.fecha_venta
    FROM libros l
    INNER JOIN ventas v ON l.titulo = v.titulo_libro
    ORDER BY v.fecha_venta DESC
  ]]>
</queryString>

```

XML

Línea 3	SELECT l.titulo, → recupera la columna titulo de la tabla libros utilizando el alias de tabla l.
Línea 4	l.precio, → recupera la columna precio de la tabla libros.
Línea 5	v.cantidad, → recupera la columna cantidad de la tabla ventas utilizando el alias de tabla v.
Línea 6	v.fecha_venta → recupera la columna fecha_venta de la tabla ventas.
Línea 7	FROM libros l → indica la tabla libros con el alias l.
Línea 8	INNER JOIN ventas v ON l.titulo = v.titulo_libro → combina las dos tablas por la columna titulo de libros y titulo_libro de ventas.
Línea 9	ORDER BY v.fecha_venta DESC → ordena las filas por fecha de venta descendente.

Los alias de tabla son nombres cortos que se asignan a las tablas para simplificar las referencias en la consulta. En lugar de escribir libros.titulo se escribe l.titulo. Los alias se declaran después del nombre de la tabla en la cláusula FROM. Los alias de columna se declaran con la cláusula AS. La combinación de alias de tabla y alias de columna permite escribir consultas complejas de forma legible. La convención en el proyecto EditorialReports es usar alias de una o dos letras para las tablas y alias descriptivos para las columnas.

TEXT

TIPOS DE JOIN

INNER JOIN:

Devuelve solo las filas con coincidencia en ambas tablas.
Ejemplo: libros que tienen al menos una venta.

LEFT JOIN:

Devuelve todas las filas de la tabla izquierda y las coincidentes de la derecha, con NULL donde no hay coincidencia.
Ejemplo: todos los libros, con o sin ventas.

RIGHT JOIN:

Devuelve todas las filas de la tabla derecha y las coincidentes de la izquierda.
Ejemplo: todas las ventas, con o sin libro asociado.

FULL OUTER JOIN:

Devuelve todas las filas de ambas tablas.
SQLite 3.44 sí soporta `RIGHT JOIN` y `FULL OUTER JOIN` (soporte incorporado desde SQLite 3.39).

Qué representa el diagrama: los tipos de JOIN y el tipo de filas que devuelve cada uno. La elección depende del informe que se quiera construir.

Por qué es relevante: permite elegir el tipo de JOIN adecuado según el informe. Los informes de ventas utilizan INNER JOIN. Los informes de libros con o sin ventas utilizan LEFT JOIN.

Resumen rápido de la teoría

- Las consultas SQL se declaran en el elemento queryString con language="sql".
- La correspondencia entre columnas y campos se establece por nombre.
- Los alias de columna permiten renombrar columnas y nombrar columnas calculadas.
- Los parámetros se referencian con \$P{} y se enlazan mediante JDBC; \$P!{} se reserva para sustitución textual controlada.
- Las funciones de agregación son SUM, AVG, COUNT, MAX y MIN.
- La cláusula GROUP BY agrupa las filas y permite aplicar agregaciones por grupo.
- La cláusula HAVING filtra los grupos después de la agregación.
- Los JOIN combinan filas de varias tablas según una condición.

Punto 3.6 — Fields

Módulo, proyecto y objetivos de aprendizaje

Módulo: 3 — Conexión a datos (3,5 horas) **Proyecto:** EditorialReports — sistema de informes empresariales para una editorial
Punto: 3.6 — Fields

Objetivos de aprendizaje

- Comprender el campo como contrato entre la consulta SQL y la plantilla JRXML.
- Establecer la correspondencia entre los tipos SQL y los tipos Java de los campos.
- Gestionar valores nulos en los campos mediante `isBlankWhenNull` y expresiones condicionales.
- Depurar errores de resolución de campos con las herramientas del entorno.
- Aplicar buenas prácticas en la declaración de campos de un informe profesional.
- Documentar los campos del informe de ventas en el proyecto EditorialReports.

Parte teórica

Bloque 1 — El campo como contrato entre consulta y plantilla

Un campo en JasperReports es la declaración que establece el contrato entre el `ResultSet` de la consulta SQL y la plantilla del informe. El campo declara un nombre y un tipo Java. El nombre debe coincidir con el nombre de una columna del `ResultSet` o con el alias de una columna calculada. El tipo Java debe ser compatible con el tipo SQL de la columna. El motor utiliza esta declaración para solicitar el valor a la fuente de datos y para convertirlo al tipo esperado. Sin la declaración del campo, el motor no puede resolver la expresión `$F{ }` que lo referencia y lanza un error de resolución.

```
<field name="titulo" class="java.lang.String"/>
```

XML

Línea 1

`<field name="titulo" class="java.lang.String"/>` → declara un campo llamado `titulo` de tipo cadena. El nombre coincide con la columna `titulo` del `ResultSet`. El tipo `java.lang.String` corresponde al tipo SQL `TEXT` de la base de datos SQLite.

El contrato del campo tiene dos condiciones. La primera es que el nombre del campo coincida con el nombre de la columna o con el alias de la columna calculada. La segunda es que el tipo Java del campo sea compatible con el tipo SQL de la columna. Si la primera condición falla, el motor no encuentra la columna y lanza `Field not found`. Si la segunda condición falla, el motor intenta la conversión y puede lanzar `ClassCastException` o producir un resultado incorrecto. La coherencia entre el contrato declarado en el JRXML y la estructura del `ResultSet` es condición necesaria para que el informe funcione.

CONTRATO DEL CAMPO

TEXT

Consulta SQL:

```
SELECT titulo, precio, paginas FROM libros
```

ResultSet:

```
columna "titulo" → TEXT
columna "precio" → REAL
columna "paginas" → INTEGER
```

Declaración en el JRXML:

```
<field name="titulo" class="java.lang.String"/>    - coincide con TEXT
<field name="precio" class="java.lang.Double"/>    - coincide con REAL
<field name="paginas" class="java.lang.Integer"/>  - coincide con INTEGER
```

El contrato se cumple en los tres casos.

Qué representa el diagrama: la correspondencia entre las columnas del `ResultSet`, sus tipos SQL y las declaraciones de campo del JRXML. El contrato se cumple cuando los nombres y los tipos coinciden.

Por qué es relevante: permite verificar de un vistazo si la declaración de campos es coherente con la consulta. La discrepancia en el nombre produce un error de resolución. La discrepancia en el tipo produce un error de conversión.

Bloque 2 — Correspondencia entre tipos SQL y tipos Java

La correspondencia entre los tipos SQL y los tipos Java no es uno a uno. Un mismo tipo SQL puede mapearse a varios tipos Java según el uso que se le vaya a dar. Un tipo `INTEGER` puede declararse como `java.lang.Integer`, `java.lang.Long` o `java.lang.Double` según la precisión necesaria. Un tipo `REAL` puede declararse como `java.lang.Double`, `java.lang.Float` o `java.math.BigDecimal`. Un tipo `TEXT` puede declararse como `java.lang.String`. Un tipo `DATE` puede declararse como `java.util.Date` o `java.sql.Date`. La elección del tipo Java determina cómo se comporta el campo en las expresiones y qué patrones se le pueden aplicar.

TEXT

CORRESPONDENCIA ENTRE TIPOS SQL Y TIPOS JAVA

Tipo SQL	Tipo Java recomendado	Patrón aplicable
INTEGER	<code>java.lang.Integer</code>	<code>#,##0</code>
BIGINT	<code>java.lang.Long</code>	<code>#,##0</code>
REAL	<code>java.lang.Double</code>	<code>#,##0.00</code>
NUMERIC	<code>java.math.BigDecimal</code>	<code>#,##0.00</code>
TEXT	<code>java.lang.String</code>	(ninguno)
DATE	<code>java.util.Date</code>	<code>dd/MM/yyyy</code>
TIMESTAMP	<code>java.util.Date</code>	<code>dd/MM/yyyy HH:mm</code>
BOOLEAN	<code>java.lang.Boolean</code>	(ninguno)
BLOB	<code>byte[]</code>	(ninguno)
NULL	(tipo declarado)	(según el tipo)

Qué representa la tabla: la correspondencia entre los tipos SQL más habituales y los tipos Java recomendados, con el patrón aplicable a cada uno. La elección del tipo Java determina el patrón que se puede aplicar.

Por qué es relevante: permite declarar los campos con el tipo Java correcto desde el primer momento y evitar conversiones innecesarias o errores de formato.

La conversión entre el tipo SQL y el tipo Java se realiza de forma automática cuando los tipos son compatibles. El motor solicita el valor al `ResultSet` mediante el método `getObject` y lo asigna al campo. Si el tipo Java declarado no coincide con el tipo del valor devuelto, el motor intenta la conversión. Una conversión entre tipos numéricos es habitual y no produce errores. Una conversión entre tipos incompatibles, como de `String` a `Integer`, produce `ClassCastException`. La conversión de `null` a un tipo primitivo produce `NullPointerException`. La coherencia entre el tipo declarado y el tipo del valor es la primera línea de defensa contra los errores de conversión.

XML

```
<field name="precio" class="java.lang.Double"/>
<field name="paginas" class="java.lang.Integer"/>
<field name="titulo" class="java.lang.String"/>
```

Línea 1	<code><field name="precio" class="java.lang.Double"/></code> → declara el campo <code>precio</code> de tipo <code>Double</code> . El patrón <code>#,##0.00</code> se puede aplicar.
Línea 2	<code><field name="paginas" class="java.lang.Integer"/></code> → declara el campo <code>paginas</code> de tipo <code>Integer</code> . El patrón <code>#,##0</code> se puede aplicar.
Línea 3	<code><field name="titulo" class="java.lang.String"/></code> → declara el campo <code>titulo</code> de tipo <code>String</code> . No se le puede aplicar ningún patrón numérico o de fecha.

Bloque 3 — Gestión de valores nulos

Un valor nulo en la base de datos se convierte en `null` en el campo Java. El motor imprime una cadena vacía o el texto `null` según la configuración del campo. La propiedad `isBlankWhenNull` del elemento `textField` controla este comportamiento: cuando su valor es `true`, el campo se muestra vacío si el valor es `null`; cuando su valor es `false`, el campo muestra el texto `null` o produce un error de formato si tiene un patrón. La propiedad se declara en el elemento `textField` junto a los demás atributos.

XML

```
<textField isBlankWhenNull="true" pattern="#,##0.00 €">
  <reportElement x="0" y="0" width="100" height="20"/>
  <textFieldExpression><![CDATA[{F{precio}}]></textFieldExpression>
</textField>
```

Línea 1	<code><textField isBlankWhenNull="true" pattern="#,##0.00 €"></code> → declara el campo con la propiedad <code>isBlankWhenNull</code> activada y el patrón numérico. Si el valor es <code>null</code> , el campo se muestra vacío. Si el valor no es <code>null</code> , se aplica el patrón.
----------------	---

La gestión de nulos también puede realizarse en la expresión del campo. Una expresión condicional que comprueba si el valor es `null` y devuelve un valor alternativo evita que el motor intente aplicar el patrón a un valor nulo. Si el resultado debe ser texto, ambas ramas del ternario deben producir texto. Por ejemplo: `{F{precio}} == null ? "Sin precio" : new java.text.DecimalFormat("#0.00 '€' ").format({F{precio}})`. Esta aproximación permite mostrar un texto descriptivo en

lugar de un espacio vacío. La elección entre `isBlankWhenNull` y una expresión condicional depende del efecto deseado. La propiedad es más concisa; la expresión condicional es más flexible.

XML

```
<textField>
  <reportElement x="0" y="0" width="100" height="20"/>
  <textFieldExpression><![CDATA[{$precio} == null ? "Sin precio" : new java.text.DecimalFormat("#0.00 '€").format($F{precio})]]></
textFieldExpression>
</textField>
```

Línea 3

la expresión devuelve siempre texto: Sin precio para null o un valor formateado con `DecimalFormat` cuando existe precio. De este modo no se mezclan `String` y `Double` en las ramas del ternario.

TEXT

GESTIÓN DE NULOS: TRES ESTRATEGIAS

1. `isBlankWhenNull="true"`
Muestra el campo vacío cuando el valor es null.
Adecuado para campos numéricos o de fecha con patrón.
2. Expresión condicional
Muestra un texto alternativo cuando el valor es null.
Adecuado cuando se quiere un texto descriptivo.
3. Valor por defecto en la consulta
Utiliza `COALESCE` o `IFNULL` para sustituir null por un valor.
Adecuado cuando el null debe tratarse como un valor concreto.

Qué representa el diagrama: las tres estrategias para gestionar valores nulos. Cada una es adecuada para un escenario distinto.

Por qué es relevante: permite elegir la estrategia correcta según el efecto que se quiera conseguir. La combinación de las tres estrategias cubre la mayoría de los casos.

Bloque 4 — Depuración de errores de resolución de campos

Los errores de resolución de campos son los más habituales cuando se trabaja con consultas SQL. El motor los detecta en el momento de la evaluación y los reporta con un mensaje que incluye el nombre del campo. Los errores más frecuentes son cuatro. El primero es el error de nombre: el campo declarado en el JRXML no coincide con el nombre de la columna del `ResultSet`. El segundo es el error de tipo: el tipo declarado en el JRXML no coincide con el tipo del valor devuelto. El tercero es el error de declaración: el campo no está declarado en el JRXML pero se utiliza en una expresión `$F{}`. El cuarto es el error de alias: la consulta utiliza un alias y el campo se declara con el nombre original de la columna.

TEXT

ERRORES DE RESOLUCIÓN DE CAMPOS

```
Error 1: nombre incorrecto
Consulta:  SELECT titulo FROM libros
Campo:    <field name="Titulo" .../>
Error:    Field not found: Titulo

Error 2: tipo incorrecto
Consulta:  SELECT precio FROM libros (REAL)
Campo:    <field name="precio" class="java.lang.Integer"/>
Error:    ClassCastException: java.lang.Double cannot be cast to java.lang.Integer

Error 3: campo no declarado
Consulta:  SELECT titulo, precio FROM libros
Expresión: $F{paginas}
Error:    Field not found: paginas

Error 4: alias incorrecto
Consulta:  SELECT precio * 1.21 AS precio_iva FROM libros
Campo:    <field name="precio" .../>
Error:    Field not found: precio
```

Qué representa el diagrama: los cuatro errores más frecuentes de resolución de campos, con el ejemplo mínimo de cada uno y el mensaje que produce el motor.

Por qué es relevante: permite diagnosticar con precisión el error y aplicar la corrección adecuada. Los cuatro errores se detectan en momentos distintos del ciclo de compilación y llenado.

La depuración de estos errores se realiza con cuatro herramientas. La primera es el panel **Problems** del entorno, que muestra los errores de compilación con el número de línea. La segunda es la vista **Console** del programa Java, que muestra la traza completa de la excepción. La tercera es la previsualización con un origen de datos real, que revela si el problema está en la declaración del campo o en la consulta. La cuarta es la inspección del `ResultSet` con un cliente SQL, que muestra los nombres y tipos reales de las columnas. La combinación de las cuatro herramientas permite localizar el error en pocos minutos.

TEXT

HERRAMIENTAS DE DIAGNÓSTICO

Panel Problems	→ Errores de compilación del JRXML
Vista Console	→ Traza completa de la excepción
Vista Preview	→ Comportamiento con datos reales
Cliente SQL	→ Nombres y tipos reales de las columnas
Pestaña Source	→ Inspección del XML generado

Qué representa el diagrama: las cinco herramientas de diagnóstico disponibles y su utilidad para depurar errores de resolución de campos.

Por qué es relevante: permite elegir la herramienta adecuada según el tipo de error. Los errores de compilación se detectan en Problems. Los errores de llenado se detectan en Console. Los errores de maquetación se detectan en Preview.

Bloque 5 — Buenas prácticas en la declaración de campos

La declaración de campos en un informe profesional sigue cuatro buenas prácticas. La primera es declarar los campos con el tipo Java que corresponde al tipo SQL de la columna y al uso que se le vaya a dar. La segunda es utilizar alias descriptivos en la consulta SQL para las columnas calculadas y declarar los campos con esos alias. La tercera es declarar todos los campos que se utilizan en las expresiones antes de escribir las expresiones. La cuarta es documentar los campos en un archivo de texto junto al proyecto. Estas cuatro prácticas reducen los errores de resolución y facilitan el mantenimiento del informe.

TEXT

BUENAS PRÁCTICAS EN LA DECLARACIÓN DE CAMPOS

1. Tipo Java coherente con el tipo SQL y con el uso.
INTEGER → Integer, REAL → Double, TEXT → String, DATE → Date.
2. Alias descriptivos en la consulta para columnas calculadas.
SUM(cantidad) AS unidades_vendidas
No: SUM(cantidad) AS col1
3. Declarar todos los campos antes de escribir las expresiones.
Evita errores de resolución en la compilación.
4. Documentar los campos en un archivo CAMPOS.md.
Facilita el mantenimiento y la incorporación de nuevos desarrolladores.

Qué representa el diagrama: las cuatro buenas prácticas en la declaración de campos. Cada práctica reduce un tipo de error.

Por qué es relevante: permite adoptar un enfoque sistemático en la declaración de campos que reduce los errores y facilita el mantenimiento.

La aplicación de estas prácticas requiere disciplina y constancia. La declaración de campos es una de las primeras tareas que se realizan al crear un informe, y los errores en esta fase se propagan a todas las expresiones que los utilizan. La buena práctica consiste en revisar la declaración de campos antes de escribir cualquier expresión y en verificar la coherencia con la consulta SQL antes de compilar el informe. La inversión de tiempo en esta fase se recupera con creces en la fase de depuración. La declaración de campos es la base sobre la que se construye el resto del informe.

TEXT

PROCESO DE DECLARACIÓN DE CAMPOS

1. Escribir la consulta SQL.
2. Ejecutar la consulta en un cliente SQL.
3. Anotar los nombres y tipos de las columnas del ResultSet.
4. Declarar los campos en el JRXML con el tipo Java correspondiente.
5. Verificar la compilación del informe.
6. Escribir las expresiones que utilizan los campos.
7. Documentar los campos en CAMPOS.md.

Qué representa el diagrama: el proceso de declaración de campos en siete pasos. La consulta se escribe antes que los campos y los campos antes que las expresiones.

Por qué es relevante: permite abordar la declaración de campos de forma sistemática y evitar errores que se propagan al resto del informe.

Resumen rápido de la teoría

- Un campo es el contrato entre la consulta SQL y la plantilla JRXML.
- El nombre del campo debe coincidir con la columna o el alias del ResultSet.
- El tipo Java del campo debe ser compatible con el tipo SQL de la columna.
- La propiedad `isBlankWhenNull` controla el comportamiento ante valores nulos.
- Las expresiones condicionales permiten mostrar textos alternativos para valores nulos.

- Los errores más frecuentes son de nombre, de tipo, de declaración y de alias.
- Las herramientas de diagnóstico son Problems, Console, Preview y un cliente SQL.
- Las buenas prácticas incluyen tipos coherentes, alias descriptivos y documentación.